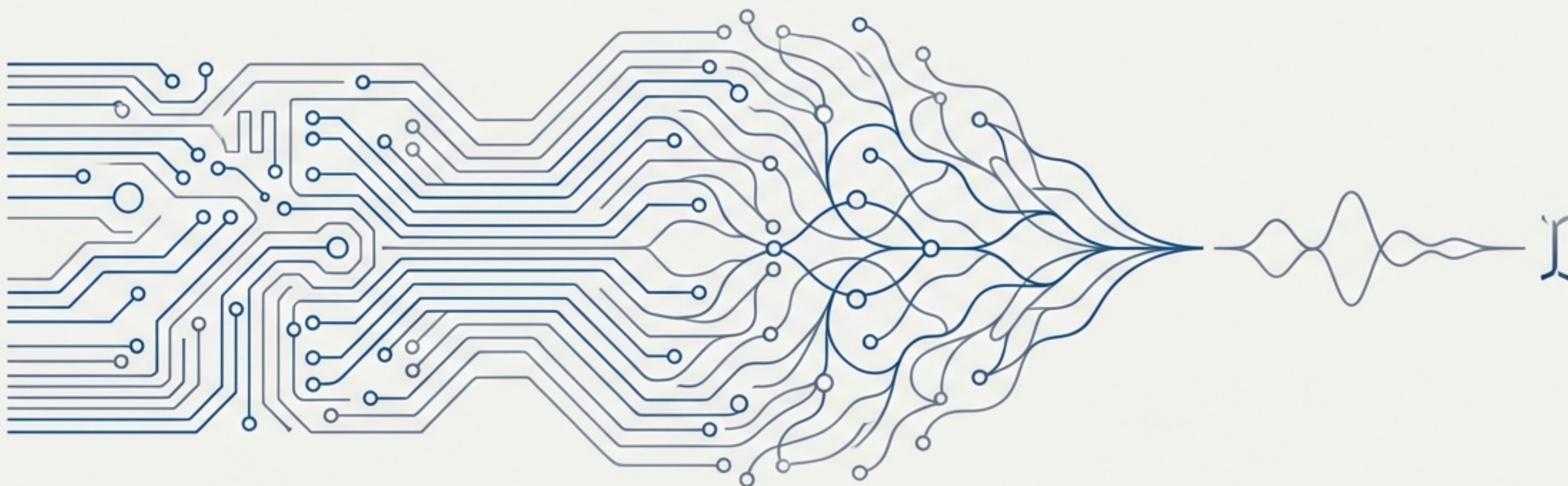


AI as the Hardware Design Co-Pilot

An Experimental Evaluation of Conversational Verilog Generation



Author: Dhirajzen Bagawath Geetha Kumaravel

Date of Experiment: September 17, 2025

Core Technology: ChipChat (GenAI)

The Experimental Setup & Ground Rules



Toolchain

- **Simulation:** Icarus Verilog (`iverilog`)
- **Interface:** Google Colab Notebook
- **AI Engine:** ChipChat, via OpenRouter API



AI Model & Parameters

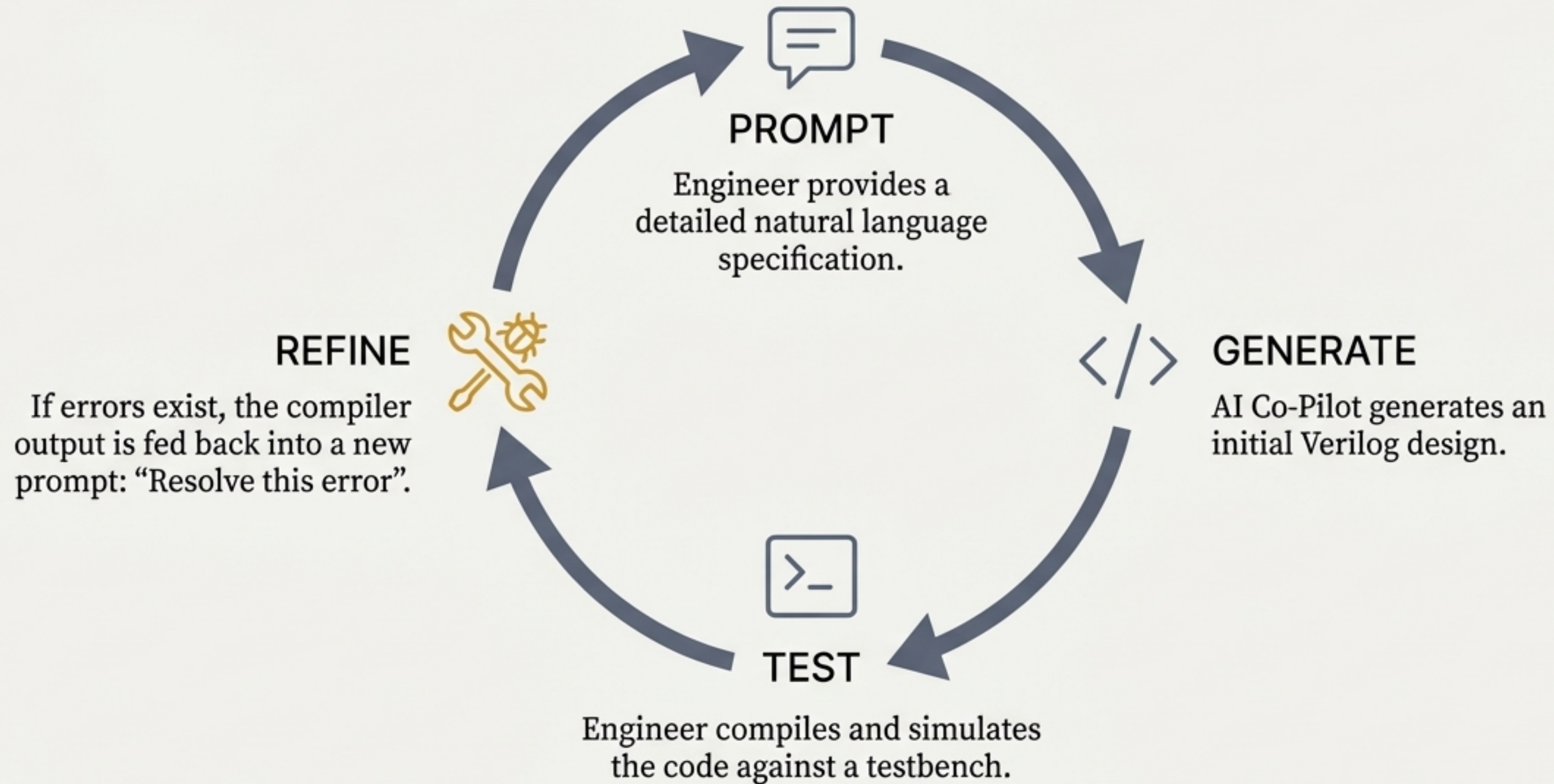
- **LLM:** `openrouter/sonoma-sky-alpha`
- **Max Tokens:** 1,700,000
- **Other Parameters:** Default (Temperature, `top_p`, etc.)



Prompting Strategy

- Start with a natural language specification.
- Explicitly constrain the output (synthesizable, no SystemVerilog).
- If errors occur, provide the **entire** error log back to the AI.

The Human-AI Interaction Loop



This feedback loop was the fundamental workflow for all five design challenges.

Test Flight 1: Binary-to-BCD Converter

Specification: ``binary_to_bcd_converter``

Inputs: ``binary_input`` (5-bits)

Outputs: ``bcd_output`` (8-bits: 4 for tens, 4 for units)

Behavior: Convert a 5-bit binary number (0-31) to its 2-digit BCD equivalent.

Iterations to Success: 3



Attempt 1: Logical Failure

The test case for input ``10`` failed. The AI's code (`{binary_input / 10, binary_input % 10}`) produced an incorrect bit-width in the concatenation.



Attempt 2: Syntax Error

The AI's attempt to fix the width (`{ (div)[3:0], (mod)[3:0] }`) was illegal syntax in Verilog.



Attempt 3: Final Fix

Introduced intermediate ``wire`s` to hold arithmetic results *before* slicing and concatenation. This satisfied the Verilog syntax and produced the correct logic.

Code Evolution: From Logical Flaw to Syntactic Fix

Initial Output (Logical Error)

```
// Fails for input 10
assign bcd_output =
{binary_input / 5'd10,
 binary_input % 5'd10};
```

Incorrect width in concatenation leads to test failure. e.g., ``10/10=1`` is not 4 bits.

Second Output (Syntax Error)

```
// Won't compile
assign bcd_output = {
    (binary_input / 10)[3:0],
    (binary_input % 10)[3:0]
};
```

Introduces illegal syntax. Cannot bit-slice an arithmetic expression directly in a continuous assignment.

Final Correct Design

```
// Correct solution
wire [4:0] tens_wire =
binary_input / 5'd10;
wire [4:0] units_wire =
binary_input % 5'd10;
assign bcd_output =
    {tens_wire[3:0],
     units_wire[3:0]};
```

Using intermediate wires resolves both the logical and syntactic issues.

Test Flight 2: Sequence Detector FSM

Specification: ``sequence_detector``

Inputs: `clk`, `reset_n`, `data` (3-bits)

Output: `sequence_found`

Behavior: Detect a specific 8-value sequence (`'001 -> 101 -> 110...`) using a state machine.

Iterations to Success: 3



Attempt 1: Elaboration Error

The AI named the output port `seq_found`, but the testbench expected `sequence_found`. A simple but critical interface mismatch.



Attempt 2: Simulation Failure

After fixing the port name, the simulation failed at cycle 8. The registered output logic created an off-by-one timing error; the `sequence_found` signal was asserted one cycle too late.



Attempt 3: Final Fix

Changed the output from a registered signal (output reg) to a purely combinatorial one (output wire), aligning the output timing with the testbench's expectation.

```
Change: sequence_found <= 1'b1;  
To: assign sequence_found = (state == 3'b111 &&  
data == 3'b101);
```


Test Flights 3 & 5: First-Try Successes



Shift Register

Spec: An 8-bit shift register with serial input, parallel output, active-low reset, and a shift enable.

Result: The AI generated a fully correct, synthesizable design using standard Verilog constructs.

Iterations: 1. Passed all test cases immediately.



Parameterizable PWM Generator

Spec: A PWM generator with configurable WIDTH, enable, and synchronous reset.

Result: The AI correctly implemented the counter logic, parameterization, and output generation.

Iterations: 1. Passed all duty cycle tests (0%, 25%, 50%, 75%, 100%, 33%) without modification.

Test Flight 4: Dice Roller

Specification: ``dice_roller``

Inputs: ``clk``, ``rst_n``, ``die_select`` (2-bits), ``roll`` trigger.

Output: ``rolled_number`` (8-bits)

Behavior: On a roll trigger, generate a pseudo-random number from 1 to N, where N is 4, 6, 8, or 20 based on ``die_select``.

Iterations to Success: 3



Attempt 1: Compilation Error

The AI used ``integer sides;`` inside the ``always`` block. This is a SystemVerilog feature and is illegal in standard Verilog.



Attempt 2: Elaboration Error

A familiar problem: the testbench expected ports named ``die_select`` and ``rolled_number``, but the AI had generated ``die_sel`` and ``rolled_num``.



Attempt 3: Final Fix

The ``integer`` declaration was removed and the logic was flattened into the ``case`` statement. The port names were corrected to match the testbench.

A Pattern of Errors Emerges



Interface Mismatches

The AI fails to perfectly match port names defined in the testbench, causing elaboration errors. This is a frequent, but easily fixed, issue.

Examples:

- `sequence_detector :`
 `(`seq_found` vs`
 ``sequence_found`)`
- `dice_roller :(`die_sel` vs`
 ``die_select`;`
 ``rolled_num` vs`
 ``rolled_number`)`



Language & Syntax Violations

The AI uses constructs that are syntactically incorrect or belong to a different language version (e.g., SystemVerilog) than requested.

Examples:

- `binary_to_bcd_converter :`
 (Illegal bit-slicing on an expression)
- `dice_roller :` (Use of
 ``integer`` declaration inside an
 ``always`` block)



Subtle Logic & Timing Flaws

The most difficult errors to debug. The code compiles but produces incorrect results due to flawed logic or timing that mismatches the testbench's expectations.

- `binary_to_bcd_converter :`
 (Incorrect bit-width in concatenation)
- `sequence_detector :` (Off-by-one error from registered output)

The Mission Data at a Glance

Design Module	Complexity	Iterations to Success	Initial Error Type	Key Learning
Binary-to-BCD Converter	Low	3	Logical (Bit-Width)	AI can miss subtle arithmetic-to-concatenation logic.
Sequence Detector	Medium	3	Interface (Port Name)	Interface & timing must be perfectly aligned with testbench.
Shift Register	Low	1	None	Excels at standard, well-defined building blocks.
Dice Roller	Medium	3	Language (SystemVerilog)	Prone to using features outside the specified language standard.
PWM Generator	Medium	1	None	Handles parameterized modules and standard logic correctly.

The Co-Pilot's Performance Review



Strengths (What it does well)

- **Rapid Prototyping:** Generates a complete, structurally sound baseline for standard modules in seconds.
- **Boilerplate & Structure:** Correctly implements module ports, FSM skeletons, and register logic.
- **Synthesizable Code:** The generated code is generally synthesizable, avoiding abstract or non-hardware constructs.

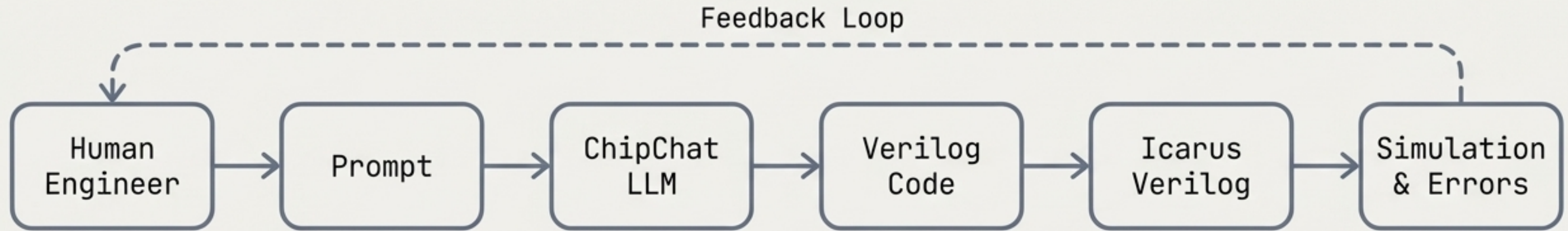


Weaknesses (Where the human pilot is essential)

- **Interface Precision:** Frequently mismatches exact port names from specifications.
- **Subtle Logic/Timing:** Can create off-by-one errors or logical flaws that only simulation can reveal.
- **Language Constraint Adherence:** May 'leak' features from other HDL languages (like SystemVerilog) into Verilog designs.
- **Error Interpretation:** Fixes errors literally, sometimes introducing new syntactic flaws in the process.

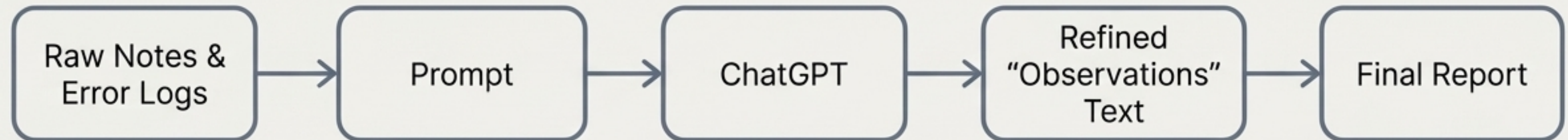
A Note on the Process: AI Co-Pilot vs. AI Scribe

The Core Experiment



This loop represents the entire hardware generation and debugging process.

The Documentation Process



ChatGPT assisted only in refining the English descriptions for the final report; it was not involved in generating or debugging the Verilog.

The Hardware Designer's AI Co-Pilot Playbook

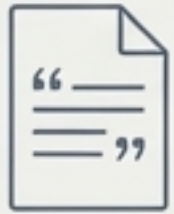
The Rules of Engagement

1



Be an Exacting Pilot: Your initial prompt is critical. Provide precise, unambiguous specifications for inputs, outputs, and behavior. Explicitly state constraints like “synthesizable Verilog-2001.”

2



Provide the Full Crash Report: Don't summarize errors. Copy and paste the *entire, verbatim* output from the compiler or simulator into the follow-up prompt.

3



Trust, but Verify Rigorously: The AI provides a starting point, not a final product. Assume there are bugs. A comprehensive testbench is non-negotiable.

4



Expect Interface & Timing Bugs: Pay close attention to port names, off-by-one timing errors, and combinatorial vs. registered output logic. These are the AI's most common blind spots.

5



Use AI to Accelerate, Not Abdicate: View the AI as a tool that eliminates boilerplate and suggests initial structures, freeing up your engineering expertise for the crucial tasks of verification, debugging, and refinement.